

LAB: FILE PATH TRAVERSAL - SIMPLE CASE

PROGRAMACIÓN WAP

JESÚS ISRAEL MARTÍNEZ ESCOBEDO

ÍNDICE

Contenido

1. INTRODUCCIÓN	2
2. OBJETIVO	2
3. HERRAMIENTAS UTILIZADAS.....	2
4. PROCEDIMIENTO	2
4.1 Objetivo	2
4.2 Contexto técnico	3
4.3 Procedimiento	3
Evidencia 1 — Solicitud original.....	4
Figura 1. Solicitud HTTP original de una imagen.	4
4.4. Análisis de la solicitud original.....	4
4.5. Alteración de la solicitud	5
Evidencia 2 — Solicitud modificada.....	5
Figura 2. Solicitud HTTP modificada mediante Burp Suite.	5
4.6. Análisis del payload	5
4.7. Análisis de la respuesta.....	6
Evidencia 3 — Respuesta del servidor	7
Figura 3. Respuesta obtenida después de modificar la ruta.	7
4.8. Resultado del laboratorio	7
Evidencia 4 — Lab Solved	8
Figura 4. Confirmación de resolución del laboratorio.	8
5. CONCLUSIÓN.....	8

1. INTRODUCCIÓN

El presente documento describe la realización del laboratorio File Path Traversal, simple case, correspondiente a la plataforma PortSwigger Web Security Academy.

El laboratorio tiene como finalidad identificar una vulnerabilidad de Path Traversal en una aplicación web que utiliza un parámetro controlado por el usuario para recuperar imágenes de productos. Mediante la modificación de dicho parámetro se analiza si es posible acceder a archivos ubicados fuera del directorio destinado originalmente para las imágenes.

Para realizar el análisis se utilizó Burp Suite, específicamente la herramienta Repeater, con la finalidad de interceptar, modificar y reenviar una solicitud HTTP.

2. OBJETIVO

El objetivo del laboratorio es identificar y explotar una vulnerabilidad de Path Traversal que permita acceder a un archivo fuera del directorio esperado por la aplicación.

La finalidad específica del laboratorio consiste en recuperar el contenido del archivo:

/etc/passwd

3. HERRAMIENTAS UTILIZADAS

Burp Suite

Se utilizó Burp Suite para analizar y modificar las solicitudes HTTP generadas por la aplicación. La herramienta Repeater permitió reenviar manualmente las solicitudes después de modificar el parámetro vulnerable.

PortSwigger Web Security Academy

Se utilizó el entorno de laboratorio proporcionado por PortSwigger Web Security Academy para realizar la práctica en un entorno controlado y verificar posteriormente su resolución.

4. PROCEDIMIENTO

4.1 Objetivo

El objetivo de este laboratorio es identificar y explotar una vulnerabilidad de Path Traversal presente en el mecanismo utilizado por la aplicación para recuperar imágenes de productos. La finalidad consiste en modificar el parámetro utilizado para especificar

el archivo solicitado y comprobar si es posible acceder a un recurso ubicado fuera del directorio esperado, específicamente al archivo `/etc/passwd`.

4.2 Contexto técnico

La aplicación utiliza el endpoint `/image` para recuperar las imágenes asociadas a los productos. La solicitud HTTP observada durante el funcionamiento normal de la aplicación utiliza el parámetro `filename` para indicar el nombre del archivo que debe ser recuperado.

La solicitud original identificada fue:

`GET /image?filename=16.jpg HTTP/2`

El valor `16.jpg` corresponde a una imagen almacenada en el directorio destinado por la aplicación para este tipo de recursos.

El punto de interés de seguridad se encuentra en que el nombre del archivo es proporcionado mediante un parámetro controlado por el cliente. Si la aplicación utiliza este valor directamente para construir una ruta en el sistema de archivos y no aplica una validación o restricción adecuada, un atacante puede intentar manipular dicha ruta mediante secuencias de navegación de directorios.

4.3 Procedimiento

Para analizar el comportamiento de la aplicación se utilizó Burp Suite, específicamente la herramienta Repeater, con el objetivo de modificar y reenviar la solicitud HTTP de manera controlada.

El procedimiento realizado fue el siguiente:

1. Se accedió al laboratorio correspondiente de PortSwigger Web Security Academy.
2. Se ingresó a una página de producto para identificar la solicitud utilizada para recuperar su imagen.
3. Se identificó el endpoint `/image` y el parámetro `filename`.
4. Se observó que la solicitud original utilizaba el archivo `16.jpg`.
5. La solicitud fue enviada a Burp Suite Repeater para poder modificarla.
6. Se sustituyó el valor de `filename` por una secuencia de Path Traversal:
`../ ../etc/passwd`
7. Se envió la solicitud modificada al servidor.
8. Se analizó la respuesta HTTP obtenida.
9. La respuesta devolvió el contenido del archivo `/etc/passwd`.

10. Finalmente, se verificó en Web Security Academy que el laboratorio había sido marcado como Lab Solved.

Evidencia 1 — Solicitud original

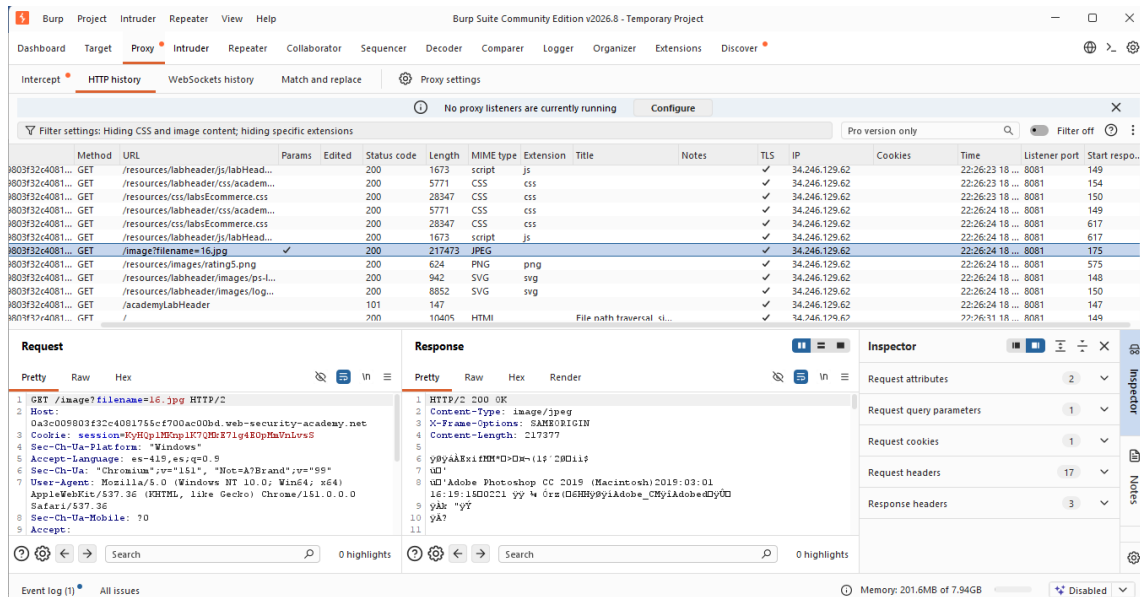


Figura 1. Solicitud HTTP original de una imagen.

Se observa la solicitud HTTP generada por la aplicación para recuperar una imagen de producto mediante el endpoint `/image`. La petición utiliza el parámetro `filename`, cuyo valor original es `16.jpg`. Este parámetro determina el recurso que será solicitado por el servidor y constituye el punto de entrada que será analizado para comprobar la existencia de una vulnerabilidad de Path Traversal.

4.4. Análisis de la solicitud original

La solicitud original identificada fue:

```
GET /image?filename=16.jpg HTTP/2
```

En esta solicitud, el endpoint `/image` recibe mediante el parámetro `filename` el nombre del recurso que debe recuperar.

La presencia de este parámetro resulta relevante desde el punto de vista de seguridad debido a que su valor es controlado por la solicitud del cliente. Si el servidor utiliza directamente dicho valor para construir una ruta del sistema de archivos sin aplicar controles adecuados, existe la posibilidad de manipular la ubicación del recurso solicitado.

4.5. Alteración de la solicitud

Para comprobar si el parámetro podía ser manipulado, se modificó únicamente su valor, sustituyendo el nombre de la imagen por:

../../../../etc/passwd

La solicitud resultante fue:

GET /image?filename=../../../../etc/passwd HTTP/2

La modificación aprovecha la secuencia ../, utilizada para representar el desplazamiento hacia el directorio padre dentro de una ruta de archivos.

En este caso, las tres secuencias ../ intentan salir progresivamente del directorio en el que la aplicación espera encontrar las imágenes y posteriormente acceder a /etc/passwd.

Evidencia 2 — Solicitud modificada

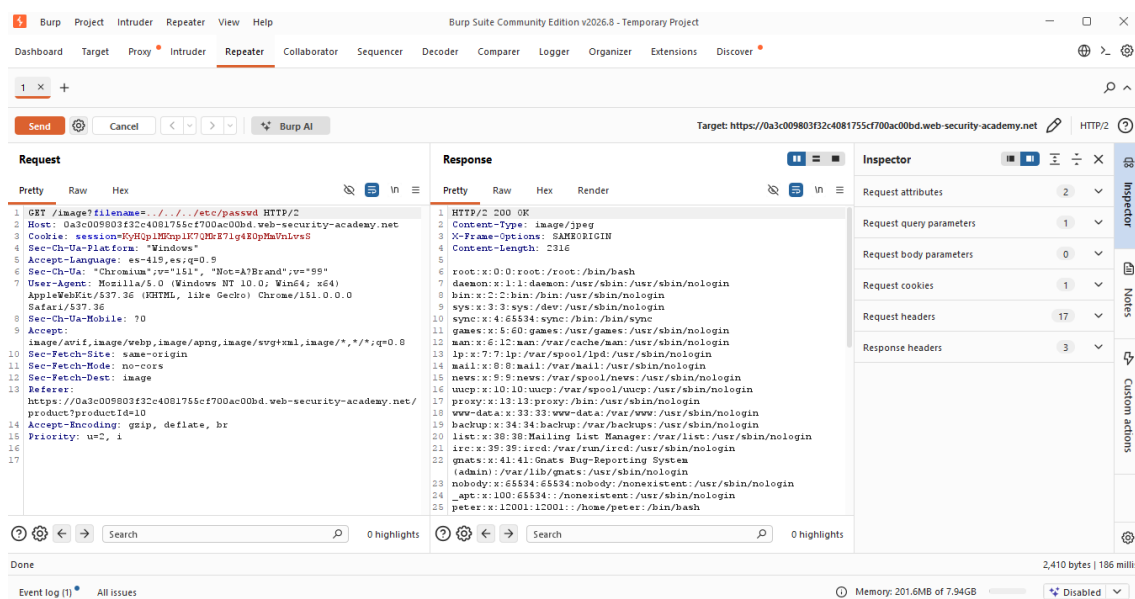


Figura 2. Solicitud HTTP modificada mediante Burp Suite.

Se modifica el parámetro filename de la solicitud original, sustituyendo el nombre de la imagen por la secuencia ../../etc/passwd. El payload utiliza tres secuencias de retroceso de directorio (../) para intentar salir del directorio destinado al almacenamiento de imágenes y acceder al archivo /etc/passwd. La modificación permite comprobar si el servidor procesa directamente la ruta proporcionada por el usuario.

4.6. Análisis del payload

El payload utilizado fue:

../../etc/passwd

Su funcionamiento se basa en la interpretación de .. como el directorio padre dentro de una ruta del sistema de archivos.

Cada secuencia ../ permite desplazarse un nivel hacia arriba en la estructura de directorios. Por lo tanto:

../

representa un nivel superior,

mientras que:

../..

representa dos niveles y:

../../

representa tres niveles superiores.

Después de realizar estos desplazamientos, la ruta continúa con:

/etc/passwd

El resultado esperado es que la aplicación deje de utilizar el directorio destinado originalmente para las imágenes y termine resolviendo una ruta que apunta al archivo /etc/passwd.

El hecho de que el servidor haya devuelto el contenido de dicho archivo demuestra que el valor controlado por el usuario fue utilizado para acceder al sistema de archivos sin una restricción efectiva del directorio permitido.

4.7. Análisis de la respuesta

La respuesta obtenida después de enviar la solicitud modificada fue:

HTTP/2 200 OK

Content-Type: image/jpeg

Sin embargo, el cuerpo de la respuesta contenía información correspondiente al archivo /etc/passwd, incluyendo entradas como:

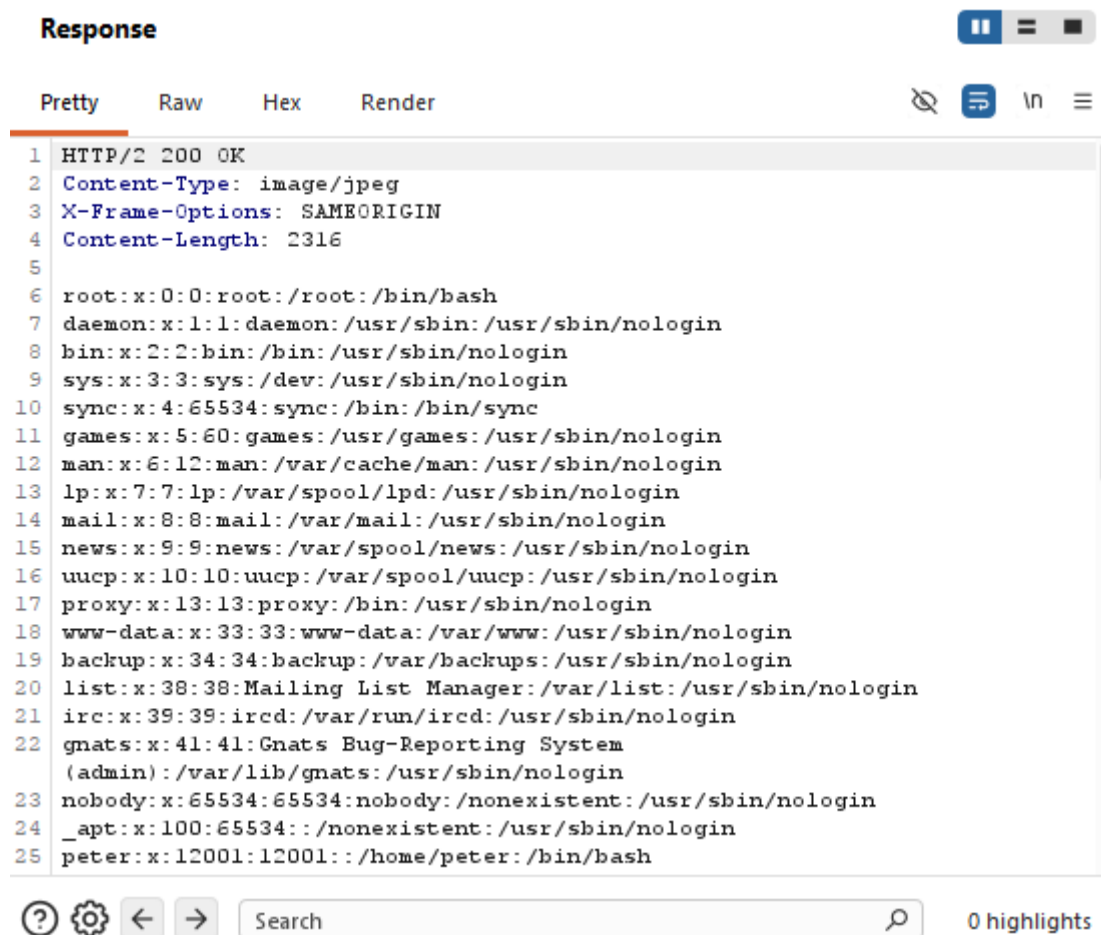
root:x:0:0:root:/root:/bin/bash

daemon:x:1:1::/usr/sbin:/usr/sbin/nologin

Este comportamiento demuestra que el servidor procesó exitosamente la ruta manipulada y devolvió el contenido del archivo solicitado.

Resulta especialmente relevante que la respuesta mantuviera el tipo de contenido asociado originalmente a una imagen mientras el cuerpo contenía texto perteneciente a un archivo del sistema. Esto evidencia que el endpoint no restringió adecuadamente el recurso al directorio esperado de imágenes.

Evidencia 3 — Respuesta del servidor



```
Response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System
(admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001::/home/peter:/bin/bash
```

Figura 3. Respuesta obtenida después de modificar la ruta.

La respuesta HTTP contiene el contenido del archivo `/etc/passwd`, incluyendo las entradas correspondientes a las cuentas existentes en el sistema. El acceso a este archivo confirma que la aplicación procesó la ruta proporcionada en el parámetro `filename` sin restringir adecuadamente el acceso al directorio destinado a las imágenes.

4.8. Resultado del laboratorio

La explotación permitió recuperar correctamente el contenido de `/etc/passwd` mediante la manipulación del parámetro `filename`.

Posteriormente, la plataforma PortSwigger Web Security Academy confirmó que el laboratorio había sido completado correctamente mediante el indicador **Lab Solved**.

Evidencia 4 — Lab Solved

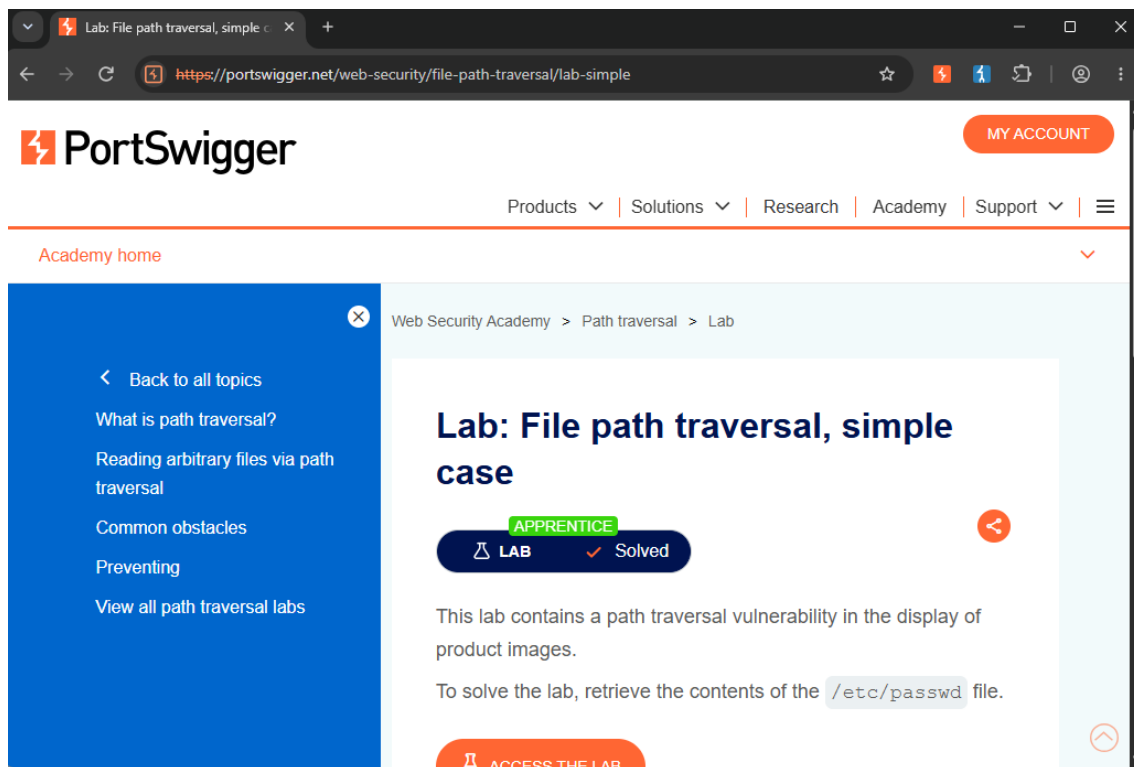


Figura 4. Confirmación de resolución del laboratorio.

La plataforma Web Security Academy confirma que el laboratorio fue resuelto correctamente. La resolución se obtuvo mediante la explotación de la vulnerabilidad de Path Traversal, utilizando el parámetro filename para acceder al archivo `/etc/passwd` fuera del directorio destinado al almacenamiento de imágenes.

5. CONCLUSIÓN

El laboratorio permitió comprobar de manera práctica el funcionamiento de una vulnerabilidad de **File Path Traversal**. El problema se originó porque un parámetro controlado por el cliente permitió influir directamente en la ubicación del archivo que debía ser recuperado.

Mediante la secuencia `../../` fue posible desplazarse fuera del directorio destinado a las imágenes y solicitar el archivo `/etc/passwd`. La respuesta obtenida confirmó que el servidor procesó la ruta manipulada y permitió acceder al contenido del archivo.

Este ejercicio demuestra la importancia de validar y restringir las rutas de archivos proporcionadas por el usuario. Una aplicación segura debe evitar que los parámetros controlados por el cliente puedan modificar arbitrariamente la ubicación de los recursos que serán procesados por el servidor.